PURE-SOFTWARE 3-MEMBER PARALLEL BLUEPRINT

PROJECT APRICORN

STGNN PCB Routing Intelligence — Parallel Team Division & Vibe-Coding Agent Guardrails

DOC-ID: APR-SW3-2026-V2
MODE: 100% PURE SOFTWARE
CORE: STGNN + WAVEFRONT A*
EXCLUDED: PHYSICSFORGE & RL-PPO

1. Pure-Software Scope & 3-Member Parallel Execution Strategy

TEAM-ARCH // 01

Project Apricorn is restructured into a **100% pure-software Python/PyTorch/TypeScript routing system** with zero hardware dependencies, zero external C++ CAD binaries, and **complete removal of PhysicsForge** and legacy RL-PPO. To enable three developers to build the system in parallel using AI-assisted ("vibe coding") workflows while maintaining true sequential data dependencies (`Member 1 → Member 2 → Member 3`), all three engineers code against a locked, immutable **Shared Data Contract (`apricorn/contracts.py`)** established on Day 1.

HOW SEQUENTIAL DEPENDENCY WORKS IN PARALLEL WITHOUT BLOCKING

Member 2's STGNN model depends directly on Member 1's graph tensors (`GraphDataContract`), and Member 3's Negotiated Router & Web Visualizer depends directly on both Member 1's graph and Member 2's STGNN cost predictions (`RoutingCostContract`). During parallel coding, a lightweight 25-line deterministic dummy generator (`apricorn/mocks.py`) emits exact-shape synthetic contracts so all three members build and test simultaneously, merging with a single switch flip on Integration Day.

2. Role Assignments & Inter-Member Dependency Chain

DEPENDENCY-GRAPH // 02



Figure 2.1: Coupled 3-Member Software Pipeline with Strictly Scoped Directory Ownership

ZERO FILE OVERLAP // ZERO MERGE CONFLICTS

Team Member	Exclusive Directory	Upstream Dependency (Input)	Downstream Deliverable (Output)
Member 1 (Data & Graph Lead)	`apricorn/data_graph/`	Raw synthetic or KiCad S-expression board netlist JSON (`BoardSpecJSON`).	Produces validated `GraphDataContract` & DRC R-tree validator consumed by Member 2 & Member 3.
Member 2 (STGNN AI Lead)	`apricorn/stgnn_engine/`	**Depends on Member 1:** Consumes `GraphDataContract` + epoch overflow vector `h_t`.	Produces `RoutingCostContract` (edge weights `W_edge` & net priority `P_net`) consumed by Member 3.
Member 3 (Router & UI Lead)	`apricorn/router_ui/`	**Depends on M1 & M2:** Consumes M1's `GraphDataContract` + M2's `RoutingCostContract`.	Produces DRC-clean `RoutedBoardResult`, feeds congestion back to M2, and renders Web UI.

3. Strict Vibe-Coding Anti-Hallucination Protocol (All AI Agents) AGENT-GUARDRAILS // 03

Because all three team members are building their modules using AI coding assistants ("vibe coding" in Cursor, Windsurf, Claude Code, or Copilot), unguarded prompts will result in mismatched tensor dimensions, invented library calls, or re-introduction of deprecated modules. Every team member **MUST prepend the Master System Prompt below** to their AI agent's `.cursorrules` or system instructions before generating any code.

MANDATORY MASTER SYSTEM PROMPT FOR ALL 3 MEMBERS' AI AGENTS (COPY-PASTE VERBATIM)

```
[SYSTEM DIRECTIVE — ZERO ASSUMPTIONS & ZERO HALLUCINATION MODE]
1. STRICT CONTRACT OBEDIENCE: You are coding one isolated module of Project Apricorn. You MUST
import all shared data structures exclusively from `apricorn/contracts.py`. NEVER invent, rename,
reorder, or alter dictionary keys, dataclass fields, or tensor shapes (`x_nodes: [N, 12]`,
`edge_index: [2, E]`, `edge_attr: [E, 6]`).
2. FORBIDDEN MODULES: NEVER implement or reference `PhysicsForge`, thermal solvers, impedance
calculators, RL/PPO (`gym`, `stable-baselines3`), hardware drivers, or external C++ binaries. This
is a 100% pure-software geometric + STGNN routing system.
3. DIRECTORY LOCK: You may ONLY create or edit files inside your assigned folder
(`apricorn/data_graph/`, `apricorn/stgnn_engine/`, or `apricorn/router_ui/`). NEVER modify another
team member's directory or `apricorn/contracts.py`.
4. PINNED LIBRARIES ONLY: Use ONLY Python 3.11 standard library, `numpy`, `scipy`, `shapely`,
`rtree`, `networkx`, `torch`, `torch_geometric`, and `fastapi`. NEVER hallucinate non-existent
functions or unapproved third-party packages.
5. STOP-AND-ASK RULE: If any requirement, edge case, or tensor dimension is ambiguous, STOP
IMMEDIATELY and ask the user for clarification. Do NOT guess or silently assume behavior.
```

4. The Immutable Day-0 Interface (`apricorn/contracts.py`) SHARED-CONTRACT // 04

Before splitting into parallel streams, the team commits this single 45-line file to the Git repository root. It acts as the mathematical handshake uniting Member 1, Member 2, and Member 3.

```
# FILE: apricorn/contracts.py (LOCKED DAY-0 — DO NOT MODIFY WITHOUT ALL 3 MEMBERS) from
dataclasses import dataclass from typing import Dict, List, Tuple import numpy as np
@dataclass(frozen=True) class GraphDataContract: """Produced by MEMBER 1 (data_graph) ->
Consumed by MEMBER 2 (stgnn) & MEMBER 3 (router_ui)""" board_id: str grid_shape: Tuple[int,
int, int] # (Layers L, Grid_X, Grid_Y) x_nodes: np.ndarray # Shape: (N, 12) float32 -> [x, y,
z, is_pin, is_obs, cap, dem, h_overflow, net_id_norm, deg, dist_bnd, layer_dir] edge_index:
np.ndarray # Shape: (2, E) int64 -> Directed spatial adjacency list (src_node, dst_node)
edge_attr: np.ndarray # Shape: (E, 6) float32 -> [length, dx, dy, dz_via, base_cost,
congestion_ratio] net_demands: Dict[int, List[int]] # {net_id: [ordered list of terminal
node_indices to connect]} obstacle_polygons: List[Tuple[int, List[Tuple[float, float]]]] #
[(layer_idx, [(x,y), ...])] @dataclass(frozen=True) class RoutingCostContract: """Produced by
MEMBER 2 (stgnn_engine) -> Consumed by MEMBER 3 (router_ui)""" epoch: int edge_weights:
np.ndarray # Shape: (E,) float32 strictly >= 1.0 (predicted traversal cost per directed edge)
net_priorities: Dict[int, float] # {net_id: priority_score in (0.0, 1.0)} -> higher routes
first hidden_state: np.ndarray # Shape: (N, 64) float32 -> persistent temporal GRU state H_t
@dataclass(frozen=True) class RoutedBoardResult: """Produced by MEMBER 3 (router_ui) ->
Consumed by Web UI & Member 2 Feedback Loop""" epoch: int routed_paths: Dict[int, List[int]]
# {net_id: [sequence of node_indices forming routed trace]} edge_overflow: np.ndarray #
Shape: (E,) float32 -> overflow count per edge fed back to Member 1/2 for epoch t+1
drc_violations: int # Total remaining geometric clearance/short violations (0 = converged)
wirelength_mm: float # Total routed octilinear trace length via_count: int # Total interlayer
z-transitions
```

DETERMINISTIC MOCK GENERATOR (`APRICORN/MOCKS.PY`) FOR DAY-1 INDEPENDENCE

Alongside `contracts.py`, a helper script `apricorn/mocks.py` provides `make_mock_graph(L=2, X=32, Y=32, nets=15) -> GraphDataContract` and `make_mock_costs(E) -> RoutingCostContract` using seeded NumPy arrays (`np.random.default_rng(42)`). Member 2 and Member 3 import these mocks to develop and test their code on Day 1 while Member 1 builds the real graph engine.

5. Member 1 Specification: PCB Graph, DRC & Dataset Architect

MEMBER-01 // DATA-GRAPH

Role & Ownership: Member 1 owns `apricorn/data_graph/`. Member 1 transforms raw 2D/3D board layouts into the canonical `GraphDataContract`, enforces pure-software geometric Design Rule Checking (DRC) via `shapely` and `rtree`, and generates synthetic routed boards so Member 2 can train the STGNN.

- **Deliverable 1.1 — Board Parser & Synthetic Generator (`parser_gen.py`):** Parses simplified JSON board descriptions or S-expression `.kicad_pcb` pad/keepout blocks, and includes a procedural generator producing 500 solvable multi-layer board instances (with ground-truth A* routes) for Member 2's training pipeline.
- **Deliverable 1.2 — 3D Octilinear Grid-Graph Extractor (`graph_builder.py`):** Discretizes the board into a $L \times X \times Y$ GCell mesh with 8 planar compass directions ($0°$, $45°$, $90°$, $135°$, ...) plus vertical Z-via edges, populating all 12 node features (`x_nodes`) and 6 edge features (`edge_attr`) defined in `GraphDataContract`.
- **Deliverable 1.3 — Pure-Software Geometric DRC Engine (`drc_engine.py`):** Uses `shapely.geometry.LineString` buffered by trace half-width + clearance ($w/2 + c_{\min}$) indexed in an `rtree` spatial index to detect trace-to-trace shorts, pad collisions, and keepout violations in $<5ms$.

> **PROMPT FOR MEMBER 1'S AI AGENT (Append to Master Guardrail Prompt):** You are coding ONLY inside `apricorn/data_graph/`. Implement `parser_gen.py`, `graph_builder.py`, and `drc_engine.py`. 1. Your `build_graph(board_json, prev_overflow=None) -> GraphDataContract` function MUST output exact `np.float32` and `np.int64` arrays matching `apricorn/contracts.py`: `x_nodes` shape `(N, 12)`, `edge_index` shape `(2, E)`, `edge_attr` shape `(E, 6)`. 2. Every planar GCell node `(l, x, y)` connects to at most 8 planar neighbors on layer `l` (orthogonal length `1.0`, diagonal length `1.4142`) and vertical via neighbors `(l±1, x, y)` (base_cost `4.0`). Ensure `edge_index` contains both `(u, v)` and `(v, u)`. 3. Implement `check_drc(graph: GraphDataContract, paths: Dict[int, List[int]], clearance: float) -> int` using `shapely` and `rtree` only. Do NOT use any external CAD CLI or hardware tool.

6. Member 2 Specification: STGNN Core AI Engine Lead

MEMBER-02 // STGNN-AI

Role & Ownership: Member 2 owns `apricorn/stgnn_engine/`. Member 2's code is strictly dependent on Member 1's `GraphDataContract` (using `mocks.py` until Member 1's generator is merged) and builds the Spatio-Temporal Graph Neural Network that predicts edge traversal weights and net routing order for Member 3.

- **Deliverable 2.1 — Spatial-Temporal Network (`model.py`):** Implements a 3-layer edge-conditioned `GATv2Conv` spatial encoder ($12 \rightarrow 64$ channels, 4 attention heads, `edge_dim=6`) followed by a custom `GraphConvGRUCell` that updates temporal hidden state $H_t \in \mathbb{R}^{N \times 64}$ across negotiation epochs t.
- **Deliverable 2.2 — Dual Decoding Heads (`heads.py`):** (1) *Edge Cost Head:* Concatenates `[H_t[src], H_t[dst], edge_attr]` ($64+64+6 = 134$ dims) through a 2-layer MLP with `1.0 + Softplus(x)` activation to guarantee strictly positive edge weights $W_{\text{edge}} \geq 1.0$. (2) *Net Priority Head:* Pools terminal node embeddings per `net_id` and applies `Sigmoid` to output `net_priorities`.
- **Deliverable 2.3 — Training & Inference Wrapper (`predictor.py`):** Provides `STGNNPredictor.predict(graph: GraphDataContract, prev_hidden: Optional[np.ndarray], epoch: int) -> RoutingCostContract` that runs CPU/GPU inference and returns a locked `RoutingCostContract`.

> **PROMPT FOR MEMBER 2'S AI AGENT (Append to Master Guardrail Prompt):** You are coding ONLY inside `apricorn/stgnn_engine/`. Implement `model.py`, `train.py`, and `predictor.py` using `torch` and `torch_geometric`. 1. Your model MUST accept `GraphDataContract` from `apricorn/contracts.py` (node features `dim=12`, edge features `dim=6`) and optional `hidden_state` `(N, 64)`. 2. In `predictor.py`, implement `STGNNPredictor.predict(...) -> RoutingCostContract`. Guarantee via `1.0 + torch.nn.functional.softplus(...)` that every value in `edge_weights` is strictly `>= 1.0` and has shape `(E,)` `np.float32` with zero `NaN` or `Inf` values. 3. Include a deterministic pre-trained checkpoint saver/loader so

7. Member 3 Specification: Negotiated Router & Interactive Web UI Lead

MEMBER-03 // ROUTER-UI

Role & Ownership: Member 3 owns `apricorn/router_ui/`. Member 3's module depends directly on **both** Member 1's `GraphDataContract` (plus `check_drc`) and Member 2's `RoutingCostContract`. Member 3 implements the negotiated congestion Rip-Up-and-Reroute A* engine and the browser-based PCB layout & congestion visualizer.

- **Deliverable 3.1 — Multi-Pin Octilinear Wavefront A* Router (`astar_router.py`):** Connects multi-terminal nets in descending order of Member 2's `net_priorities`. Traverses Member 1's `edge_index` using composite edge cost $C_e^{(t)} = (b_e + \alpha \cdot W_{edge}[e]) \times p_e^{(t)} \times h_e^{(t)}$, with octilinear Euclidean heuristic admission and 45° bend penalties.
- **Deliverable 3.2 — Closed-Loop Negotiation Controller (`negotiator.py`):** Runs up to $T = 20$ epochs. At each epoch t: calls Member 2's `STGNNPredictor`, rips up overflowing nets, re-routes them via `astar_router.py`, updates historical congestion $h_e^{(t)} = h_e^{(t-1)} + \eta \cdot Overflow(e)$, and calls Member 1's `check_drc()` until `drc_violations == 0`.
- **Deliverable 3.3 — Pure-Software Web Dashboard (`server.py` + `static/index.html`):** Lightweight FastAPI server + HTML5 Canvas/SVG frontend showing live epoch-by-epoch trace routing, STGNN attention/cost heatmaps, DRC violation badges, and wirelength/via telemetry.

> **PROMPT FOR MEMBER 3'S AI AGENT (Append to Master Guardrail Prompt):** You are coding ONLY inside `apricorn/router_ui/`. Implement `astar_router.py`, `negotiator.py`, and `server.py` + `static/index.html`. 1. Your router MUST consume `GraphDataContract` (from Member 1) and `RoutingCostContract` (from Member 2) defined in `apricorn/contracts.py`, and return `RoutedBoardResult`. 2. During parallel development, allow toggling `--use-mocks` so `negotiator.py` can run against `apricorn/mocks.py` before Member 1 and Member 2 merge their code. 3. For multi-pin nets (`len(terminals) > 2`), implement Prim-A* tree expansion: route from pin 0 to pin 1, then treat all nodes along the existing subtree as zero-cost sources when routing to pin `k`. Never invent edges not present in `edge_index`.

8. End-to-End Sequential Data Handoff Matrix

DATA-FLOW // 08

The table below traces the exact software execution sequence during a single routing request once all three members merge their directories. Every step relies on the preceding member's output.

Step	Executing Owner	Function Invoked	Exact Data Handoff & Dependency
Step 1	**Member 1**	`build_graph(board_json, h_t)`	Parses board layout and constructs `GraphDataContract` (`x_nodes[N,12]`, `edge_index[2,E]`, `edge_attr[E,6]`).
Step 2	**Member 2** *(Depends on M1)*	`stgnn.predict(graph, H_prev, t)`	Ingests Member 1's `GraphDataContract`, runs GATv2 + GC-GRU, and emits `RoutingCostContract` (`edge_weights[E]`, `net_priorities`).
Step 3	**Member 3** *(Depends on M1+M2)*	`route_epoch(graph, costs, h_t)`	Orders nets by Member 2's `net_priorities` and runs Wavefront A* over Member 1's graph weighted by Member 2's `edge_weights`.
Step 4	**Member 1 + 3** *(Closed Loop)*	`check_drc(graph, routed_paths)`	Member 3 passes candidate paths to Member 1's R-tree DRC verifier; if overflow > 0, feeds `edge_overflow[E]` back to Step 1/2 for Epoch $t+1$.

9. Final Zero-Conflict Merge Protocol & Unified Entrypoint

MERGE-PROTOCOL // 09

Because each member works inside an isolated Python subpackage (`data_graph`, `stgnn_engine`, `router_ui`) and strictly obeys `apricorn/contracts.py`, merging all three branches into `main` produces **zero Git file conflicts**. Integration consists simply of replacing the mock import with the live class imports in the 30-line master runner below.

```
# FILE: apricorn/main.py — FINAL MERGE SCRIPT (Connects Member 1 + Member 2 + Member 3)
from apricorn.data_graph.parser_gen import load_or_generate_board # Member 1 from
apricorn.data_graph.graph_builder import build_graph, update_overflow # Member 1 from
apricorn.data_graph.drc_engine import check_drc # Member 1 from
apricorn.stgnn_engine.predictor import STGNNPredictor # Member 2 from
apricorn.router_ui.negotiator import NegotiatedRouter # Member 3 def
run_apricorn_pipeline(board_path: str, max_epochs: int = 20): board_spec =
load_or_generate_board(board_path) graph = build_graph(board_spec) # Step 1: Member 1 builds
GraphDataContract predictor = STGNNPredictor(checkpoint_path="checkpoints/stgnn_v1.pt") #
Step 2: Member 2 initializes STGNN router = NegotiatedRouter(drc_fn=check_drc) # Step 3:
Member 3 binds Member 1's DRC engine hidden_state = None for epoch in range(1, max_epochs +
1): costs = predictor.predict(graph, hidden_state, epoch=epoch) # M1 Graph -> M2 STGNN ->
RoutingCostContract hidden_state = costs.hidden_state result = router.step_epoch(graph,
costs, epoch=epoch) # M1 Graph + M2 Costs -> M3 Wavefront A* if result.drc_violations == 0
and result.edge_overflow.sum() == 0: break graph = update_overflow(graph,
result.edge_overflow) # M3 Overflow -> M1 Graph feature update return result
```

10. Individual Pre-Merge Verification Checklist & Gate Tests

MERGE-GATES // 10

Before any member opens a Pull Request to merge into `main`, their local AI coding agent must run `pytest` and pass their individual contract verification gate below:

Team Member	Automated Pre-Merge Test Command	Mandatory Contract & Functional Pass Criteria
Member 1 Gate	`pytest tests/test_m1_graph.py`	Verifies `x_nodes.shape == (N, 12)`, `edge_attr.shape == (E, 6)`, bidirectional `edge_index` symmetry, valid terminal indices in `net_demands`, and accurate R-tree collision detection on overlapping segments.
Member 2 Gate	`pytest tests/test_m2_stgnn.py`	Passes `make_mock_graph()` through `STGNNPredictor.predict()`; asserts `edge_weights.shape == (E,)`, `np.all(edge_weights >= 1.0)`, `hidden_state.shape == (N, 64)`, and zero `NaN/Inf` tensors.
Member 3 Gate	`pytest tests/test_m3_router.py`	Passes `make_mock_graph()` + `make_mock_costs()` into `NegotiatedRouter`; asserts every net path starts/ends at exact terminals, respects obstacle nodes, and renders cleanly on `GET /api/board_state`.
Combined Team Gate	`pytest tests/test_e2e_merge.py`	Executes `run_apricorn_pipeline()` on a 4-layer, 40-net benchmark board; achieves **100% net connectivity** and **0 DRC violations** within ≤ 15 negotiation epochs in under $3.0\ seconds$.

SUMMARY OF TEAM WORKFLOW FROM DAY 1 TO FINAL DEMO

Step 1 (Hour 1): Commit `apricorn/contracts.py` and `apricorn/mocks.py` to Git and distribute the Master Anti-Hallucination Prompt to all three members. **Step 2 (Days 1–5):** Member 1 builds `data_graph/`, Member 2 builds & trains `stgnn_engine/`, and Member 3 builds `router_ui/` in parallel against the contract. **Step 3 (Final Day):** Merge all three branches, run `python -m apricorn.router_ui.server`, and demonstrate live STGNN-guided multi-layer PCB routing in the browser.